

YOURSTORY



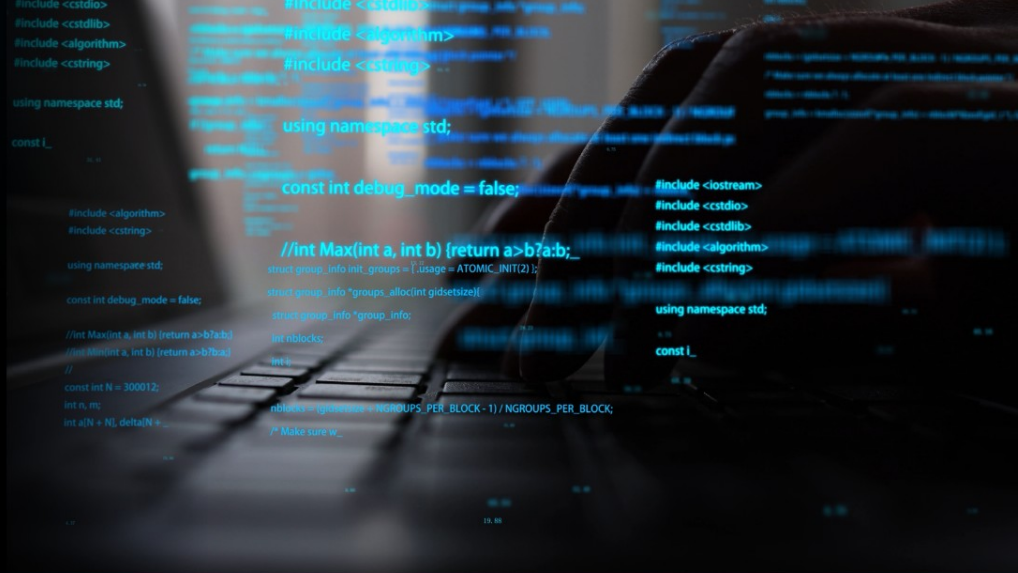
H2S



Coco CLI

HACKATHON

GCC Edition



Team Name : [Team name]

Team Leader Name : [Team leader name]

Team Size : [Team size]

Problem Statement : [Track 3 - Predictive Maintenance & OEE Command Center](#) · [Plant Brain](#)

1. Problem Brief

Sensor data can tell you that a machine is drifting. It can't tell you what fixed it last time.

Business problem

Unplanned downtime, and repeat failures where the right fix was already known, but only in a senior technician's head or a scribbled shift note. Illustrative: ~₹1 crore/year lost output for a 12-asset plant.

Target user

Maintenance planners and technicians in discrete manufacturing. Persona: Anjali, maintenance planner (fictional plant, synthetic data).

Pain point → improvement

Today: alarms, phone calls, digging through old work orders; a bearing-only fix recurred in 9 days. With Plant Brain: the anomaly arrives with the cited known fix, the "don't do this" warning, parts risk and the right technician. A human approves, and the closing note becomes new knowledge.

Industry / domain context

Manufacturing OEE and predictive maintenance. OT sensor data sits apart from maintenance context, and an ageing workforce is retiring across GCC and Indian plants.

Anjali

Maintenance planner, Deccan Precision Components, Pune (fictional)

3

production lines

12

critical assets

8

technicians

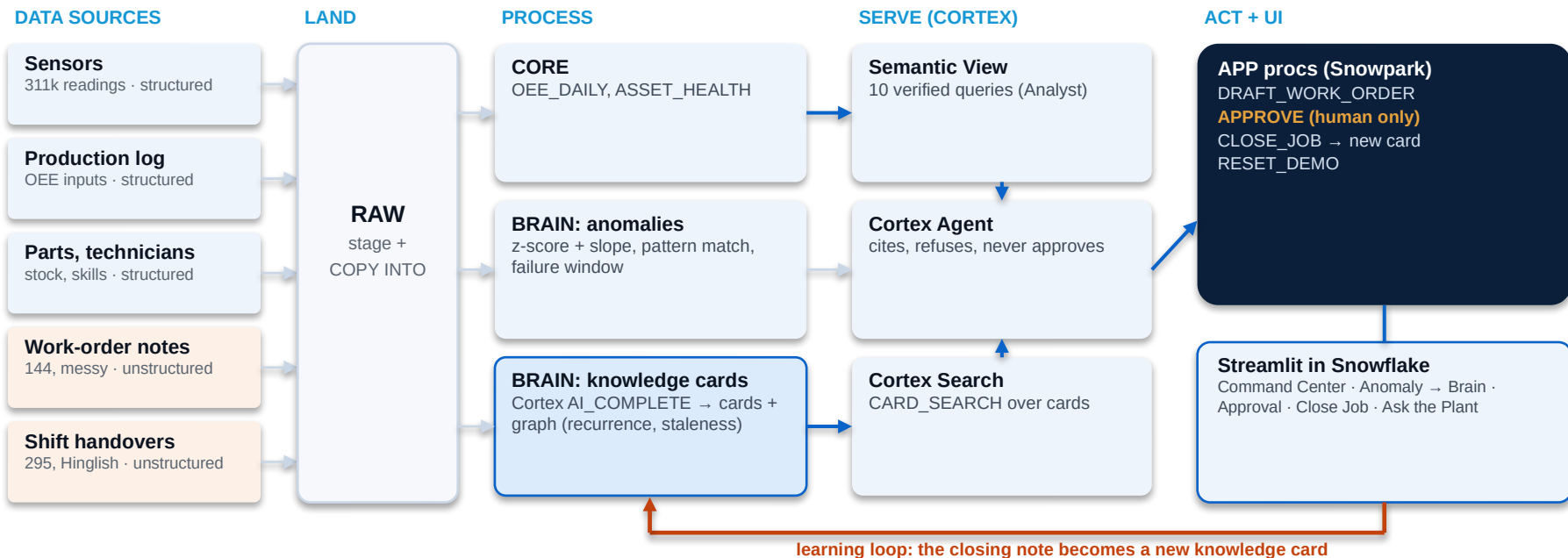
2

retire in 2027

"When Press P3 acts up, everyone calls Ravi. Ravi retires next year."

2. Architecture Diagram

All Snowflake. Structured + unstructured sources → Brain → Cortex services → human-approved actions.



CoCo CLI skills: card-extractor (extraction rules → AI_COMPLETE prompt + mock) and work-order-drafter (drafting rules → DRAFT_WORK_ORDER). **Modular**: one SQL file per box (00–09); one core Python package runs in Streamlit, the stored procedures and a local mock that executes the same SQL on DuckDB.

3. Impact Statement

Measured on the synthetic golden scenario; business outcomes are illustrative targets.

25/25

golden questions answered correctly

naive keyword-RAG baseline: 10/25

5/5

out-of-scope questions correctly refused

baseline: 3/5

12/12

expected knowledge cards found in messy, Hinglish notes

incl. the one clear note, HN-0163

21-54 h

failure-window estimate for P3

contains the true ~24 h remaining

Local mock run, no LLM (retrieval, routing, graph, guard). Cortex numbers: eval/run_eval.py --snowflake.

Measurable outcomes (targets)

- Time to find the known fix: phone calls and digging → under a minute, on screen
- Repeat failures avoided: the 9-day bearing recurrence is flagged before work starts
- Parts risk caught early: BRG-6312 at 1 in stock, 21-day lead time

Scalability

- Snowflake-native: no data leaves the account; XS warehouse, auto-suspend
- Incremental: one AI_COMPLETE call per new note, cached by text hash
- Same schema per plant; cards shareable across sister plants

Beyond the demo

- Knowledge survives retirements: every closed job adds cards
- Staleness alerts when an asset changes (basic flag live)
- Voice notes and an MCP server for technician apps

The solution: one loop: detect, recall, act, learn

Demo scenario: Press P3 has failed this way three times; on the last day the same pattern starts again.

01

Anomaly detected

P3 vibration +9%, temperature following: CRITICAL (matches 4 past failures)

02

Brain recall

Ravi's fix: bearing AND grease LG-2 → HT-3, cited from handover note HN-0163

03

Failure window

Estimate 21–54 h to threshold, shown as a range, never one number

06

Close job → new card

The technician's closing note becomes knowledge (SUPERSEDES the failed fix)

05

Human approves

The agent can draft but never approve; agent approvals are refused

04

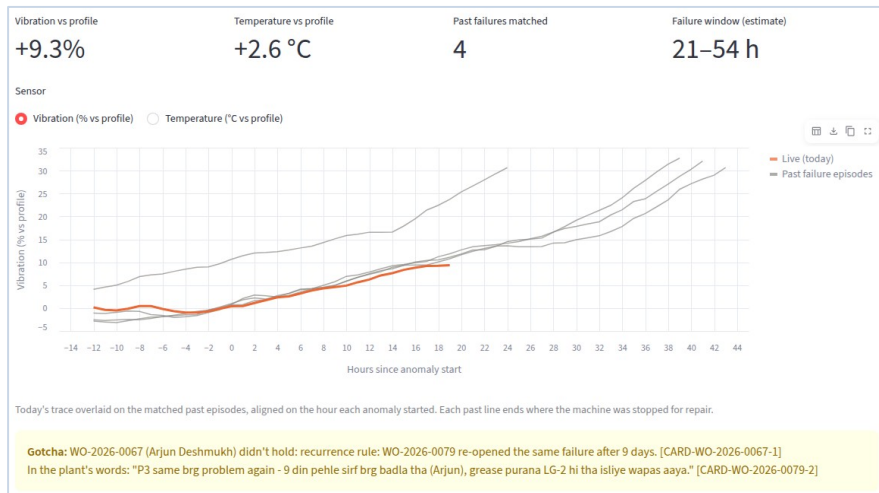
Drafted work order

Cited fix, "Do NOT" list, parts at risk, suggested technician

The gotcha the graph found: the junior's July note says "trial run ok". Only the link to the work order 9 days later shows that the bearing-only fix failed, so Plant Brain warns against repeating it.

The product: Streamlit in Snowflake, five pages

Real screens from the built app (local mock backend, same code as Snowflake).



Anomaly → Brain: today's P3 trace (orange) over past failures, failure window and the 9-day gotcha

Suspected failure mode: BEARING_WEAR, matching 4 past episode(s) on this asset

Failure window (ESTIMATE): 21-54 h to threshold (vibration_rms $\geq 1.35 \times$ baseline)

Recommended action

- Fix = replace bearing AND switch grease from LG-2 to HT-3 high-temp grade, flush old grease fully. [CARD-HN-0163-2]
 - Same fix applied before: WO-2026-0021 by Ravi Kulkarni [CARD-WO-2026-0021-1]
 - Same fix applied before: WO-2026-0079 by Ravi Kulkarni [CARD-WO-2026-0079-1]
 - Same fix applied before: WO-2026-0116 by Ravi Kulkarni [CARD-WO-2026-0116-1]

Do NOT

- P3 same brg problem again - 9 din pehle sirf brg badla tha (Arjun), grease purana LG-2 hi tha isliye wapas aaya. [CARD-WO-2026-0079-2]
- Only bearing change does NOT hold - today it came back in 9 days after bearing-only job. [CARD-HN-0163-3]
- Ravi bol raha tha lube change karo warna phir se aayega - note kar lo sab. [CARD-HN-0165-1]
- Repeat WO-2026-0067: recurrence rule: WO-2026-0079 re-opened the same failure after 9 days [CARD-WO-2026-0067-1]

Known symptom pattern: IMPORTANT P3 NOTE (pls read): when P3 vibration starts creeping up slowly over 1-2 days and then temperature also starts rising, it is the DE bearing (6312) failing. [CARD-HN-0163-1]

Parts

- BRG-6312 (Deep groove ball bearing 6312 (press DE)): 1 in stock, reorder point 2, lead time 21 d Δ AT RISK: reorder now
- LUB-HT3 (High-temp bearing grease HT-3, 18 kg drum): 1 in stock, reorder point 2, lead time 10 d Δ AT RISK: reorder now

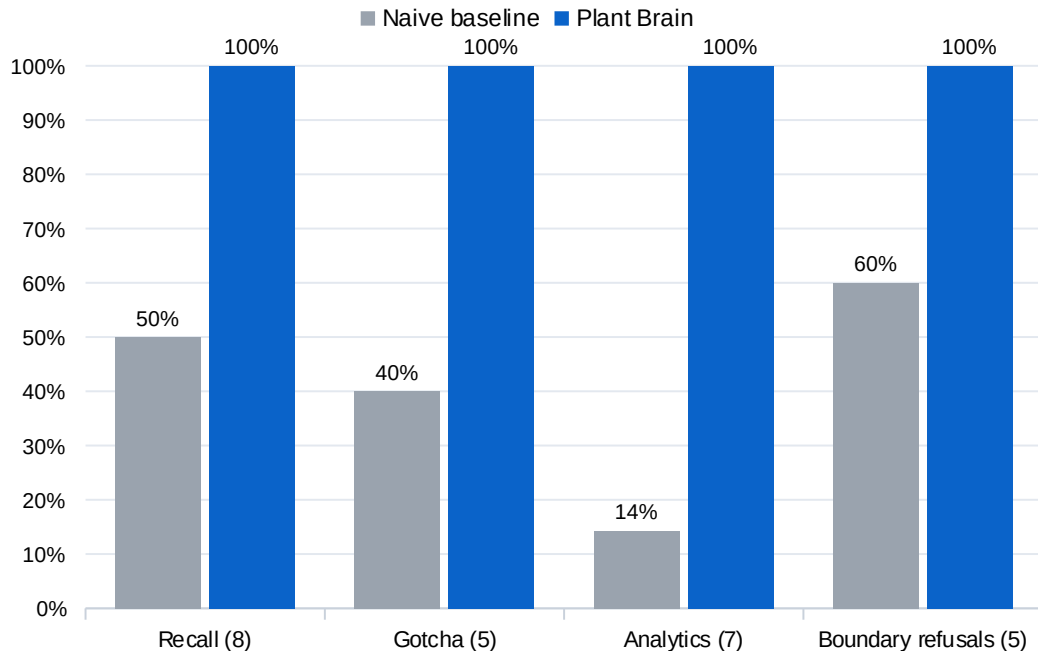
Suggested technician: Ravi Kulkarni (T01), authored 4 resolved fix(es) for this pattern; retiring 2027, pair with a junior to transfer knowledge

Work Order Approval: cited fix, "Do NOT" list, parts at risk, Ravi suggested; a human approves

Also: Command Center (OEE by line, active anomalies) · Close Job (closing note → new card + graph links) · Ask the Plant (cited answers, verified-query analytics, refusals).

Evaluation: Plant Brain vs a naive baseline

25 golden questions + golden-anomaly retrieval. Baseline = keyword search over raw notes (no cards, graph, verified queries or guard).



Overall: 25/25 vs 10/25

- Golden anomaly hit@3: both find HN-0163, but only Plant Brain ranks Ravi's explicit fix first
- Citations: 20/20 answerable answers cite card ids or a verified query

Honest caveats

- Local mock run without an LLM: measures retrieval, routing, graph and guard, not fluency
- Tuned and tested on the same synthetic scenario we designed
- First run 22/25: the eval caught 3 bugs, fixed and logged in coco/PROMPTS.md

How it's built: CoCo CLI + Snowflake-native services

Prompts, skills and guard rails are in the repo, so judges can see exactly how CoCo was used.

How CoCo CLI was used

Skills: card-extractor and work-order-drafter encode the rules the agent and the procedures follow

Prompt log: coco/PROMPTS.md records each phase, what was produced, and every bug our own checks caught

Guard rails: AGENTS.md: synthetic data only, GA features, seed=42, human approval, cost discipline

Verification: generator --verify, 25 PASS/FAIL checks, smoke test, headless browser run, eval

Repo: [\[repo link\]](#) (public before submission)

Run it: make setup test app (local, 5 min) · scripts/deploy_snowflake.py · JUMPSTART.md

Cortex AI_COMPLETE

JSON-schema card extraction, incremental

Cortex Search

CARD_SEARCH with attribute filters

Semantic View

Cortex Analyst + 10 verified queries

Cortex Agent

Search + Analyst + draft tool

Snowpark procs

draft / approve / close job

Streamlit in Snowflake

5-page app

Tasks + monitor

backfill task, 20-credit cap

Plain SQL

OOE, anomalies, graph views

YOURSTORY



1125

Coco CLI

HACKATHON

GCC Edition

THANK YOU



```
#include <algorithm>
#include <string>

using namespace std;

const int debug_mode = false;

#include <algorithm>
#include <string>

using namespace std;

const int debug_mode = false;

//int Max(int a, int b) {return a>b?a:b;}
//int Min(int a, int b) {return a<b?b:a;}
//
const int N = 300012;
int n, m;
int a[N + N], delta[N + N];

// Make sure w
```